Choosing a cross-platform technology for a Hangman app

IDATT2506 project exercise

Tri Le

October 2026

What the app has to do

The app lets one person play Hangman on a phone. It picks a word, hides the letters, takes guesses one at a time, and ends when the word is out or the attempts are gone. The game itself is small. Two other requirements decide the technology.

The solution has to work in Norwegian and English, both of them fully, with an info page in the chosen language that the player can open at startup and during a game. And the gallows grows with every wrong guess, so the app paints a shape of its own instead of only arranging text and buttons.

Those two pull in different directions. Translation is a question of how a framework handles resources, locale resolution and plural forms. Painting is a question of how it gets pixels onto the screen. Hangman is small enough to build in any of the five candidates, so the useful comparison is which one holds up on both counts.

The five candidates

Ionic builds the interface out of web technology, HTML, CSS and JavaScript, with Angular, React or Vue on top. The app then runs inside the platform web view, which Ionic describes plainly: both the iOS and Android SDKs "provide Web Views which render any Ionic app, while still allowing for full Native SDK access" (Ionic, n.d., Native Access section). Capacitor bridges to the device APIs.

Apache Cordova is the same architecture, older. The code is HTML5, CSS3 and JavaScript, the web view can own the entire interface, and everything else arrives as plugins that a new project does not have until you add them, core ones included (Apache Cordova, n.d.). Ionic with Capacitor is where this approach is maintained now, which makes Cordova hard to argue for in a new project.

React Native is JavaScript or TypeScript with React, and it renders real platform views. The current architecture dropped the old serialising bridge for JSI, which lets JavaScript hold a reference to a C++ object and call straight into it without serialisation (Meta, n.d.). Fabric renders, Hermes executes. Native views, driven from a JavaScript runtime.

Flutter is Dart, and it draws its own interface. It "has its own implementations of each UI control, rather than deferring to those provided by the system" (Google, n.d., Widgets section), and release builds "are compiled directly to machine code, whether Intel x64 or ARM instructions, or to JavaScript if targeting the web" (Google, n.d., Architectural overview section). The renderer, Impeller, ships inside the app.

Kirigami is KDE's framework for applications that adapt between desktop and phone. It sits on QML and Qt Quick Controls 2, components in QML with C++ underneath, and it runs on

Plasma Mobile, desktop Linux, Android, macOS and Windows (KDE Community, n.d.).

Translation

Flutter treats this as part of the framework. Strings live in ARB files, one per locale, and the toolchain generates a typed class from them with plural and placeholder syntax built into the format. The locale comes from the device unless the app overrides it, and the generated class also exposes the list of supported locales, so the two cannot drift apart. The toolchain will also write out any key a translation is missing, which turns "both languages are complete" from a claim into something the build checks. In this app the labels, the plural forms and the info page all come from there, while the per-language alphabet is kept in the code beside the word lists, because it drives the keyboard and the key filter rather than being text anyone reads.

React Native has nothing equivalent in the box. Translation means picking a library, usually i18next with a locale module. It works, but the file format, the plural rules and the fallback behaviour all become decisions you have to make and then keep consistent. Ionic inherits whatever the web framework underneath it offers, and the web has good options, so the capability is there without being uniform. Cordova gives you nothing and you bring your own. Kirigami uses Qt's translation system, which is mature, though its tooling is built for desktop work.

On translation alone: Flutter first, Ionic and Kirigami close behind, React Native fine with extra work, Cordova last.

Redrawing quickly

The order changes when the screen has to repaint fast. A web view goes through a browser engine, so an animation competes with layout, style recalculation and the one JavaScript thread driving it. That is where Ionic and Cordova are weakest, and it is why I would not pick either for an action game. React Native does better, because the views are native and JSI removed the per frame serialisation cost, but the logic still runs in a JavaScript runtime. Kirigami sits on Qt Quick, which has a real scene graph and is genuinely quick, at the price of QML plus C++ and the weakest Android support of the five. Flutter compiles to machine code and owns the pipeline from end to end.

All of that is me reasoning from how the things are built, so it is fair to ask whether anyone has measured it. Tollin and Lidekrans (2023) built the same app twice, once in React Native and once in Flutter, and compared CPU use, memory use and janky frames over three interaction tests on an Android phone. Flutter came out ahead on all three in their tests. They are careful about what that is worth: the gaps were "not significant enough to suggest that one is inherently better than the other" (p. 9), and they point at developer experience and community support as things that probably matter more. So the measurement agrees with the architecture argument and declines to settle the question, which is about what I would expect from two frameworks this mature.

Hangman needs none of this. Six repaints of a stick figure costs nothing anywhere. The point is that the choice should still hold if the next app in the same stack is a game, and only one candidate is good at both ends.

Language, typing and compilation

Dart is statically typed with sound null safety, and release builds compile ahead of time to ARM machine code. TypeScript on React Native or Ionic gives you static types while you write, over a runtime that is dynamically typed, so a type error that slips past the compiler shows up on the device instead. QML is dynamically typed and the typed half of a Kirigami program lives in

C++, which splits one app across two languages. Plain JavaScript on Cordova has neither types nor compilation.

For an exercise where someone else runs my code on an emulator, an error caught at build time is worth more than one caught by whoever is holding the phone. That favours Dart, and Kirigami's C++ half only if I accept writing in two languages.

How the five compare

	Ionic	Cordova	React Native	Flutter	Kirigami
Translation	good, from the web framework	bring your own	library, not framework	built in, generated	Qt, mature
Redrawing	web view, weakest	web view, weakest	native views, JS logic	owns the pipeline	Qt Quick, quick
Typing	TypeScript over JS	plain JS	TypeScript over JS	Dart, sound null safety	QML plus C++
Compiled	no	no	no, JS runtime	yes, ahead of time	yes, the C++ half
Android support	good	good	good	good	its weakest target
App size	small	small	small	largest, ships a renderer	middling

The last row is where Flutter loses. Bringing your own renderer means shipping it, so the code that draws the interface rides along inside the app instead of being borrowed from the phone. The release build of this app is 47.9 MB with all three ABIs in it, 17.0 MB cut down to one. For hidden letters and six lines that is a lot of megabytes, and on a metered connection it would be the first thing anyone asked me about.

What I did not do

I read documentation and one study. I did not build Hangman five times. Doing that would tell me more about what each one is like to work in than any amount of reading, and it does not fit in the day this part of the project is supposed to take.

There are things I never weighed. How big and how healthy each ecosystem is. Whether the tooling is pleasant after the first week. What you get for testing without reaching for a package, which is funny to leave out given how much testing this app ended up with. Read the choice below as the best call I could make from documentation and one measurement.

What I chose

Flutter. It is the only one of the five that is strong at both ends of what this app needs: translation handled by the framework with generated, typed access to the strings, and a rendering pipeline it owns and compiles to machine code. Static typing and ahead of time compilation move errors from runtime to build time. One widget set means the Norwegian and English layouts, keyboard alphabet included, come out of the same code instead of two platform variants.

The cost is worth writing down. Flutter is a separate SDK I had to install, and Dart is a language I had not written a line of before this project, so there was a real ramp. React Native would have cost me neither, since I already write TypeScript. I picked against my own familiarity because

the requirements pointed the other way, which is the whole point of doing this comparison before writing code instead of after.

React Native is the runner up and a defensible answer. Native views, no bridge cost, a serious language. It loses on two things. Its translation story is a library decision rather than a framework guarantee, and the logic runs in a JavaScript runtime, which is the first layer I would have to think about if this app ever had to hold a steady frame rate. Ionic and Cordova lose on the web view, and Cordova again because Capacitor replaced it. Kirigami loses on the language split and on Android being its least supported target.

References

Apache Cordova. (n.d.). *Overview*. Retrieved October 12, 2026, from https://cordova.apache.org/docs/en/latest/guide/overview/index.html

Google. (n.d.). *Flutter architectural overview*. Retrieved October 12, 2026, from https://docs.flutter.dev/resources/architectural-overview

Ionic. (n.d.). *Fundamentals*. Retrieved October 12, 2026, from https://ionicframework.com/docs/core-concepts/fundamentals

KDE Community. (n.d.). *Kirigami*. Retrieved October 12, 2026, from https://community.kde.org/Kirigami

Meta. (n.d.). *About the New Architecture*. Retrieved October 12, 2026, from https://reactnative.dev/architecture/landing-page

Tollin, G., & Lidekrans, M. (2023). *React Native vs. Flutter: A performance comparison between cross-platform mobile application development frameworks* [Bachelor's thesis, Linköping University]. DiVA. https://liu.diva-portal.org/smash/record.jsf?pid=diva2:1768521